

Министерство науки высшего образования Российской Федерации
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

Факультет технологий искусственного интеллекта

Образовательная программа Искусственный интеллект в промышленности

Направление подготовки (специальность) 09.04.02 - Информационные системы и
технологии

О Т Ч Е Т

по технологическому проекту

Тема: Разработка библиотеки генерации эвристик для SSGS-планирования
с минимальной стоимостью LLM-диалога

Студент: Михалева С.Д., группа J4151

Научный руководитель: Ковальчук Михаил Андреевич

Санкт-Петербург
2026

Оглавление

Введение.....	4
Глава 1. Аналитический обзор предметной области.....	5
1.1. Языковые модели в сфере планирования.....	5
1.2. Стратегии минимизации стоимости LLM-диалога.....	6
1.3. Выводы по главе.....	7
Глава 2. Анализ ограничений первого этапа и постановка задачи.....	8
2.1. Проблема монолитной генерации.....	8
2.2. Концепция Skeleton-интерфейса.....	9
2.3. Выводы по главе.....	10
Глава 3. Разработка каркаса планировщика.....	11
3.1. Структура scheduler_skeleton.py.....	11
3.2. ProblemState: управление состоянием.....	12
3.3. Цикл SSGS в run_schedule.....	12
3.4. Поддержка пяти типов задач в ProblemFactory.....	13
3.5. Выводы по главе.....	14
Глава 4. Форматы описания задач для LLM.....	15
4.1. Три подхода к описанию.....	15
4.2. Текстовый подход: шаблон промпта.....	15
4.3. DSL-подход: полное JSON-описание.....	16
4.4. Skeleton-подход: краткое DSL и системный промпт.....	18
4.5. Выводы по главе.....	19
Глава 5. Веб-интерфейс.....	20
5.1. Архитектура app.py.....	20
5.2. Структура интерфейса.....	21
5.3. Компиляция кода от LLM.....	25

5.4. Отображение результатов.....	25
5.5. Выводы по главе.....	27
Глава 6. Экспериментальное исследование.....	28
6.1. Условия экспериментов.....	28
6.2. Токены: сравнение диалог-стоимости.....	28
6.3. Надежность: процент успешных запусков.....	29
6.4. Качество решений.....	30
6.4.1. Базовый RCPSP.....	31
6.4.2. Single Machine / JIT.....	32
6.4.3. RCPSP с невозобновляемыми ресурсами и условный RCPSP.....	33
6.4.4. Многопроектный RCPSP.....	35
6.5. Анализ генерируемых эвристик.....	35
6.6. Выводы по главе.....	36
Глава 7. План дальнейшей разработки.....	37
Заключение.....	38
Список использованных источников.....	40

Введение

В первом семестре НИР был разработан подход к автоматической генерации эвристических алгоритмов для задач планирования с ограниченными ресурсами (RCPSP) с использованием больших языковых моделей (LLM). Были реализованы два формата описания задачи для LLM: текстовый промпт и предметно-ориентированный язык DSL (Domain-Specific Language) на базе JSON. Эксперименты показали, что DSL-подход позволяет сократить диалог-стоимость (количество токенов и итераций до получения рабочего кода). Но оба подхода имеют общий недостаток: языковая модель генерирует полную функцию `solve_scheduling`, включая всю инфраструктуру планировщика – разбор данных, ресурсный учет, цикл планирования. Это приводило к высокому проценту ошибок исполнения.

В рамках текущего этапа предложен новый подход – Skeleton-интерфейс. Он представляет собой готовый каркас планировщика со всей необходимой инфраструктурой, а языковая модель генерирует лишь две функции: выбор следующей работы (`select_job_fn`) и выделение ресурсов (`allocate_fn`).

Задачи второго этапа:

1. разработка универсального каркаса планировщика для всех пяти типов задач;
2. реализация Skeleton-интерфейса и подготовка кратких DSL-описаний;
3. разработка веб-интерфейса;
4. проведение сравнительных экспериментов по трем подходам (DSL, текстовый, Skeleton).

Глава 1. Аналитический обзор предметной области

1.1. Языковые модели в сфере планирования

В последние годы автоматизированное проектирование алгоритмов перешло на новый уровень благодаря использованию больших языковых моделей (LLM) в роли генератора кода [1]. Языковые эвристики представляют собой подход, при котором LLM итеративно создает, оценивает и улучшает код алгоритмов в открытом программном пространстве, не требуя предварительного жесткого кодирования операторов человеком [2].

В процессе эволюционного синтеза LLM оперирует не параметрическими векторами, а непосредственно текстовым представлением программного кода (как правило, на языке программирования Python), который реализует приоритетные функции [3]. Ниже представлена сравнительная таблица современных эволюционных фреймворков автоматической генерации эвристик.

Таблица 1. Сравнительная таблица существующих подходов генерации эвристик.

Фреймворк	Основной поисковый механизм	Способ генерации промптов и контекста	Метод контроля корректности кода
Dynamics-Aware Solver Heuristics (DASH) [1]	Трехуровневая коэволюция механизмов поиска и расписаний выполнения	Промпты на основе трасс сходимости (tLDR) и метрик батча; при warm-start контекст из группового архива PLR	Пороговая фильтрация кандидатов по терминальному лог-остатку и tLDR с верификацией на реальных экземплярах
Reflective Evolution (ReEvo) [2]	Генетическое программирование с двухуровневой рефлексией	Промпты включают спецификацию задачи, пару родительских эвристик с явным указанием их относительной производительности и накопленные рефлексии	Выполнение сгенерированного кода на наборе экземпляров задачи. Эвристики с ошибками выполнения исключаются из популяции при отборе родительских пар

Фреймворк	Основной поисковый механизм	Способ генерации промптов и контекста	Метод контроля корректности кода
Classical Planning with LLM-Generated Heuristics [3]	Одноуровневый сэмплинг из 25 независимых вызовов LLM с температурой 1.0	Промпт содержит описание целевого домена, две задачи из тренировочного набора, два примера эвристик (Gripper, Logistics), представление состояний в Pddl и чеклист ошибок	Корректность планов гарантируется GBFS. Эвристики, упавшие с ошибкой или не решившие ни одной тренировочной задачи, отбрасываются

Общей чертой всех трех разработок является генерация кода на Python, а также верификация кандидатов через исполнение на реальных экземплярах задач.

1.2. Стратегии минимизации стоимости LLM-диалога

Основной проблемой при практическом использовании систем автоматизированного проектирования алгоритмов на базе LLM является затраты на запросы. Для оптимизации затрат на API (Application Programming Interface) применяются следующие подходы [4]:

1. Маршрутизация моделей – сокращение расходов за счет разделения запросов по уровню сложности. Простые задачи (форматирование кода, добавление docstring) перенаправляются на дешевые высокопроизводительные модели, например, Claude Haiku или GPT-4o mini. А сложные запросы (синтез новых математических концепций) обрабатываются продвинутыми версиями моделей, таких как Sonnet, GPT-4o;
2. Уплотнение контекста – сжатие истории диалога на 50-70%. Снижение объема контекста напрямую уменьшает стоимость каждого последующего шага в диалоге;

3. Кэширование промптов – постоянные системные инструкции помечаются как кэшируемые объекты на стороне API-провайдера, предоставляя скидку до 90% при повторном использовании префиксов запросов;
4. Пакетная обработка – группировка запросов в пакеты и отправление через Batch API с гарантированным временем обработки до 24 часов. Такой метод дает снижение стоимости токенов на 50%;
5. Оптимизация промптов – исключение лишних примеров, замена текстовых конструкций компактными JSON-схемами и т.д.

1.3. Выводы по главе

Проведенный аналитический обзор показал, что ключевой фактор практической применимости LLM-ориентированных систем остается управление стоимостью диалога: итеративные фреймворки (DASH, ReEvo) обеспечивают высокое качество генерируемых решений за счет обратной связи, однако требуют значительного числа обращений к модели, тогда как одноуровневый сэмплинг намеренно жертвует итеративным улучшением ради предсказуемого бюджета вызовов.

На основании изученных подходов было принято решение реализовать генерацию отдельных функций приоритизации работ и ресурсов для решения задач планирования средствами LLM. В качестве формата запросов был выбран DSL на основе JSON и точное представление всех входных переменных для функций. Такая реализация в отличие от свободного текстового описания снижает склонность модели к галлюцинациям и вероятность получения синтаксически некорректного вывода. Это согласуется со стратегией оптимизации промптов, рассмотренной в разделе 1.2.

Глава 2. Анализ ограничений первого этапа и постановка задачи

2.1. Проблема монолитной генерации

На первом этапе LLM получала описание задач и генерировала полную функцию `solve_scheduling(jobs, resources)`. Анализ результатов выявил ряд проблем.

Во-первых, генерация семантических и инфраструктурных ошибок. Так как модель вынуждена одновременно описывать и саму эвристику, и сопутствующий код, это приводит к критическим сбоям. В текстовом подходе надежность падает до 60% на сценарии условного планирования. Сгенерированный код на основе текстового и DSL представлений для задачи Single Machine (JIT) возвращал физически невозможные значения целевой функции. К тому же монолитный код часто приводил к ошибкам времени выполнения и бесконечным циклам.

Во-вторых, высокая диалог-стоимость. Количество выходных токенов растет по мере усложнения условий задачи. Если для базовых сценариев ответ составляет около 2000 токенов, то для комплексных кейсов он достигает 4300-4900. При этом логика приоритетов в содержании занимает минимальную часть, а основной массив токенов тратится на воспроизведение базовых механизмов планировщика.

В-третьих, нестабильность структуры и отсутствие жесткого контракта. Монолитная генерация характеризуется высокой дисперсией – стандартное отклонение у объема выходных токенов втрое превышает показатели каркасного подхода. Без явного ограничения на генерацию модель склонна к избыточной креативности, она пытается реализовать сложные метаэвристики (Beam search, Multi-start), которые не способна корректно написать в рамках одного запроса.

2.2. Концепция Skeleton-интерфейса

Идея Skeleton-интерфейса основана на принципе разделения ответственности. Каркас реализует все, что не является эвристической логикой: управление состоянием задачи, ресурсный учет, цикл SSGS (Serial Schedule Generation Scheme, последовательная схема генерации расписаний), вычисление целевой функции. Языковая модель реализует только то, что является собственно эвристикой: правило выбора следующей работы и правило выделения ресурсов.

Математически это соответствует следующему разложению:

$$SSGS(D, \pi, \alpha) = SKELETON(D, select_job_fn, allocate_fn)$$

где SKELETON – детерминированный каркас, π (*select_job_fn*) – функция выбора очередной работы из множества eligible-кандидатов; α (*allocate_fn*) – функция выделения ресурсов для выбранной работы; D – описание задачи (граф прецедентности, ресурсы, метахарактеристики), рассчитываемое скриптом `src/calculate_metrics.py`. При этом даже некорректный выбор работы или ресурсов обрабатывается каркасом: он подставляет самостоятельно первый доступный вариант или вызывает `advance_time()` – функцию, которая сдвигает текущее время до момента завершения ближайшей активной работы, освобождая занятые ею ресурсы и открывая новые eligible-кандидаты.

Ключевое следствие заключается в том, что объем генерируемого кода сокращается до двух небольших функций, каждая из которых решает четко сформулированную подзадачу. Это снижает как вероятность ошибок, так и число выходных токенов.

2.3. Выводы по главе

Проведенный анализ выявил ограничения монолитного подхода к генерации алгоритмов планирования с использованием больших языковых моделей. Делегирование LLM полного цикла разработки функции, включающей и инфраструктурный код, и учет ограничений, приводит к семантическим ошибкам, избыточному расходу токенов и непредсказуемости генерируемых структур.

В качестве преодоления этих проблем предложена и обоснована концепция Skeleton-интерфейса, который базируется на строгом разделении ответственности. Выделение детерминированного неизменяемого каркаса позволяет сформировать жесткую постановку задания для языковой модели.

Глава 3. Разработка каркаса планировщика

3.1. Структура scheduler_skeleton.py

Каркас реализован в файле src/scheduler_skeleton.py на языке программирования Python. Он состоит из пяти компонентов: DSLParser, датаклассы Job и Resource, ProblemState, ObjectiveCalculator и публичного API. Ниже приведена структура ключевых датаклассов.

```
@dataclass
class DSLMeta:
    project_id: str          # идентификатор задачи
    problem_type: str
    primary_metric: str      # 'Makespan' / 'Total
Earliness/Tardiness Penalty' / ...
    primary_direction: str   # 'Minimize' / 'Maximize'
    secondary_metric: str | None
    secondary_params: dict   # например, {'due_date': 106} для Case
2
    graph: dict              # nodes, edges, cpl, avg_parallelism,
...
    has_non_renewable: bool  # Case 3
    is_multi_project: bool   # Case 5
    is_single_machine: bool  # Case 2
    conditional_scheduling: bool      # Cases 3, 4
    selection_groups_present: bool     # Cases 3, 4

@dataclass
class Job:
    id: int
    duration: int
    predecessors: list[int]
    successors: list[int]
    resources_required: dict[int, int] # resource_id -> amount
    earliness_penalty: int = 0          # Case 2
    tardiness_penalty: int = 0          # Case 2
    project_id: int | None = None       # Case 5
    non_renewable_consumption: dict[int, int] = ... # Case 3
    selection_groups: list[list[int]] = ... # Cases 3, 4

@dataclass
class Resource:
    id: int
    name: str
    capacity: int          # для renewable
    r_type: str            # 'renewable' / 'non-renewable' / 'global'
    initial_stock: int     # для non-renewable
```

3.2. ProblemState: управление состоянием

ProblemState – центральный компонент каркаса. Он хранит все изменяемое состояние задачи в процессе планирования и предоставляет методы для управления им. Ниже представлены ключевые методы с реальными фрагментами реализации.

```
def eligible_jobs(self) -> list[int]:
    # Работы, все предшественники которых завершены
    return [jid for jid in self.jobs if self.is_eligible(jid)]

def can_allocate(self, job: Job, allocation: dict[int, int]) -> bool:
    for rid, amount in allocation.items():
        if amount > self.available_capacity(rid):
            return False
    return True

def advance_time(self):
    # Перематываем до ближайшего завершения активной работы
    if self.active_jobs:
        next_finish = min(self.active_jobs.values())
        self.current_time = next_finish
        self.finish_jobs_at(next_finish)

def snapshot(self) -> dict:
    # Снапшот состояния для передачи в LLM-функции
    return {
        "current_time": self.current_time,
        "completed_jobs": list(self.completed_jobs),
        "active_jobs": dict(self.active_jobs),      # job_id ->
finish_time
        "eligible_jobs": self.eligible_jobs(),
        "renewable_used": dict(self.renewable_used),
        "non_renewable_stock": dict(self.non_renewable_stock),
        "skipped_jobs": list(self.skipped_jobs),
    }
```

3.3. Цикл SSGS в run_schedule

Публичная функция run_schedule реализует Serial Schedule Generation Scheme. Ниже приведен этот цикл планирования из каркаса с сокращениями для читаемости.

```

def run_schedule(state, select_job_fn, allocate_fn, verbose=False):
    while not state.is_done():
        eligible = state.eligible_jobs()
        if not eligible:
            state.advance_time()    # нет доступных работ –
                                   # перематываем время
            continue

        snap = state.snapshot()
        job_id = select_job_fn(eligible, state.jobs, snap, meta)
        # Защита от некорректного выбора модели
        if job_id not in eligible:
            job_id = eligible[0]
        allocation = allocate_fn(job, state.resources, snap, meta)

        if state.can_allocate(job, allocation):
            state.start_job(job_id, allocation)
            # Conditional scheduling: при запуске работы ищем
            # родительские selection_groups, в которые она входит,
            # и помечаем остальных членов группы как skipped
            if meta.selection_groups_present:
                for parent_id, parent_job in state.jobs.items():
                    if parent_id not in state.completed_jobs:
                        continue
                    for group in parent_job.selection_groups:
                        if job_id in group:
                            for alt_id in group:
                                if alt_id != job_id:
                                    state.skipped_jobs.add(alt_id)
            else:
                state.advance_time()    # нет ресурсов – перематываем
время

```

Здесь реализованы два механизма защиты от некорректного кода модели: если `select_job_fn` вернула идентификатор не из списка `eligible`, каркас подставляет `eligible[0]`; если ресурсов недостаточно, каркас вызывает `advance_time()`, то есть модели не нужно обрабатывать эти случаи самостоятельно.

3.4. Поддержка пяти типов задач в ProblemFactory

`ProblemFactory` строит `ProblemState` из сырых данных инстанса. Каждый из пяти кейсов имеет собственный формат данных, который нормализуется в единую структуру датаклассов. В таблице 2 представлены особенности каждого формата.

Таблица 2. Форматы входных данных по кейсам

Кейс	Формат resources_required	Особенности структуры
Case 1 (RCPSP)	Array [{resource_id, amount}]	Стандартный граф предшествования
Case 2 (Single Machine)	Object {'1': 1}	Поле penalties с earliness/tardiness
Case 3 (RCPSP+NRR)	Object {'id': amount}	Поле resource_consumption, selection_groups, time_successors
Case 4 (Conditional RCPSP)	Object {'id': amount}	selection_groups и time_successors, только renewable
Case 5 (RCMPSP)	Array [amount1, amount2, ...] по индексам	Вложенная структура: jobs → список проектов → activities

Спецификация форматов демонстрирует разнородность исходных данных: от классических и плоских графов до многоуровневых иерархических систем. Спроектированная ProblemFactory выступает в роли универсального адаптера, изолирующего логику нормализации этих форматов. Это позволяет свести специфичные для каждого кейса структуры к унифицированным типам данных Job и Resource.

3.5. Выводы по главе

В ходе реализации третьего этапа был разработан детерминированный каркас планировщика, который представляет собой изолированное от LLM вычислительное ядро, берущее на себя рутинные вычисления планирования работ. Такой подход позволяет перейти на уровень чистой математической логики оптимизации и гарантирует корректность вычисления итоговой целевой функции.

Глава 4. Форматы описания задач для LLM

4.1. Три подхода к описанию

В рамках исследования LLM получает описание задачи в одном из трех форматов (табл. 3). Они отличаются объемом передаваемой информации, способом представления и требованиями к выходу модели.

Таблица 3. Форматы подходов для взаимодействия с LLM

Подход	Что передается в LLM	Что генерирует LLM	Папка с файлами
DSL описание	JSON с метаярхитектурами проекта и схема данных	solve_scheduling (jobs, resources)	full DSL description of projects/
Текстовое описание	Промпт на русском языке с постановкой и требованиями	solve_scheduling (jobs, resources)	text_description_of_projects/
Skeleton	Краткий JSON только с метаярхитектурами проекта	select_job_fn + allocate_fn	brief DSL description of projects/

4.2. Текстовый подход: шаблон промпта

Для текстового подхода разработан структурированный шаблон промпта. При его создании использовался инструмент автоматической оптимизации промптов OpenAI Prompt Optimizer [5], что позволило улучшить четкость формулировок и снизить неоднозначность. Шаблон имеет следующую структуру:

[Описание задачи на естественном языке]

Условия задачи:

- [список условий и особенностей]

Цель — [целевая функция].

Длина критического пути без учёта ресурсов (нижняя граница makespan):
... единиц времени.

Эвристика на вход должна принимать такие аргументы:

- `jobs: list[dict]` — список работ. Каждый элемент содержит:
- `id (int)` — уникальный идентификатор работы

- ``duration (int)`` – длительность выполнения
- ``resources_required (dict[str, int])`` – [описание формата]
- [дополнительные поля, специфичные для кейса]
- ``resources: list[dict]`` – список ресурсов. Каждый элемент содержит:
 - ``id (int)``, ``capacity (int)``, ``type (str)``

Требования к решению:

- Реализуй подходящий эвристический алгоритм, который решит эту задачу.
- Верни полный Python-код функции `solve_scheduling(jobs, resources)`.
- Если в данных не хватает критически важного контекста, не придумывай новые входные поля и не меняй контракт функции; используй только описанные поля и делай минимальные, явно отраженные в коде допущения.
- [дополнительные требования, специфичные для кейса]

Отвечай строго в JSON формате:

```
{
  "code": "полный Python код функции solve_scheduling",
  "method": "название выбранного метода",
  "reasoning": "почему выбрано именно это правило",
  "description": "краткое описание реализации"
}
```

Верни ровно указанные поля и ровно в этом порядке. Выводи только JSON без markdown и без дополнительного текста.

За счет оптимизации формулировок и введения строгого контекстного описания данный промпт обеспечивает минимизацию риска возникновения галлюцинаций со стороны модели, типизацию выходных артефактов и явную целевую фокусировку модели.

4.3. DSL-подход: полное JSON-описание

Полное DSL-описание передает LLM агрегированные метахарактеристики задачи и схему входных данных. Ниже приведен реальный DSL для Case 1 (базовый RCPSP):

```
{
  "project_id": "PSPLIB_J30_RCPSP",
  "problem_statement": {
    "constraint_types": ["Precedence", "Resource Capacities"],
    "resource_nature": "Renewable",
    "execution_mode": "Single-Mode",
    "uncertainty": "Deterministic"
  },
  "optimization_objectives": {
    "primary": {
```



```

        "metric": "Makespan",
        "direction": "Minimize"
    }
},
"graph_meta_characteristics": {
    "nodes": 32,
    "edges": 48,
    "connected_components": 1,
    "avg_out_degree": 1.5,
    "density": 0.0484,
    "cpl": 38,
    "relative_cpl": 1.2667,
    "order_strength": 0.2067,
    "num_topo_levels": 11,
    "avg_parallelism": 4.158,
    "max_parallelism": 8,
    "num_roots": 1,
    "num_leaves": 1
},
"data_schema": {
    "resources": {
        "type": "array",
        "items": {
            "id": "integer",
            "name": "string",
            "capacity": "integer",
            "type": "string (value: renewable)"
        }
    },
    "jobs": {
        "type": "array",
        "items": {
            "id": "integer",
            "predecessors": "array[integer]",
            "successors": "array[integer]",
            "duration": "integer",
            "resource_requirements": {
                "type": "array",
                "items": {
                    "resource_id": "integer",
                    "amount": "integer"
                }
            },
            "description": "List of resource consumption entries. Empty
list [] means the job requires no resources."
        },
        "name": "string (optional, only present for Supersource and
Supersink)"
    }
}
}
}

```

Метрики графа (CPL, order_strength, avg_parallelism) позволяют модели выбрать подходящее правило приоритета без знания конкретных данных

задачи. Например, высокий `order_strength` сигнализирует о жесткой структуре предшествования, при которой правила на основе критического пути (LFT, MSLK) более эффективны.

В сравнении с текстовым представлением задач, которое накладывает скрытую нагрузку по семантическому разбору, структурированный JSON-формат в свою очередь выступает в роли предобработанного вектора признаков.

4.4. Skeleton-подход: краткое DSL и системный промпт

Для Skeleton-подхода разработаны краткие DSL-описания, не содержащие схемы входных данных – модели она не нужна, так как она не занимается разбором данных. Краткий DSL для Case 5 (многопроектный RCPSP):

```
{
  "project_id": "RCMPSP_MULTI_PROJECT",
  "problem_statement": {
    "problem_type": "RCMPSP",
    "constraint_types": ["Precedence", "Resource Capacities", "Cross-Project
Resource Sharing"],
    "resource_nature": "Renewable",
    "execution_mode": "Single-Mode",
    "uncertainty": "Deterministic"
  },
  "optimization_objectives": {
    "primary": {
      "metric": "Total Makespan (All Projects)",
      "direction": "Minimize"
    }
  },
  "graph_meta_characteristics": {
    "nodes": 184,
    "edges": 332,
    "connected_components": 2,
    "avg_out_degree": 1.804,
    "density": 0.0099,
    "cpl": 88,
    "cpl_note": "lower bound without resource constraints; equals max CPL
across both projects",
    "relative_cpl": 0.4889,
    "order_strength": 0.083,
    "num_topo_levels": 15,
    "avg_parallelism": 5.818,
    "max_parallelism": 12,
    "num_roots": 2,
    "num_leaves": 2
  }
}
```

```
}  
}
```

Системный промпт для Skeleton-клиента содержит полное описание интерфейса каркаса: сигнатуры двух функций, структуры датаклассов Job и Resource, состав снапшота и объект DSLMeta. Таким образом, prompt tokens составляют в среднем 1200-1500 (против 575-755 для полного DSL, но выходных токенов получается существенно меньше).

4.5. Выводы по главе

Текстовый подход выступает в роли традиционного интерфейса на естественном языке. Несмотря на оптимизацию и жесткие требования к JSON-ответу, данный формат несет высокую когнитивную нагрузку на модель по семантическому разбору условий, что сохраняет риски галлюцинаций.

Полный DSL-подход переводит описание задачи в строго типизированный формат JSON. Так исключается лингвистическая избыточность и предоставляется модели вектор метаяктеристик графа, позволяя механизмам внимания напрямую сопоставлять топологию проекта с релевантными правилами приоритетов.

Skeleton-подход максимизирует эффективность за счет избавления модели от необходимости знать полную схему входных данных. Баланс вычислительной нагрузки смещается в сторону системного промпта, фиксирующего строгий интерфейс взаимодействия с исполняемым каркасом.

Глава 5. Веб-интерфейс

5.1. Архитектура app.py

Интерактивный веб-интерфейс (рис. 1) реализован в файле `src/app.py` [6] на базе Streamlit. Для запуска достаточно выполнить:

```
streamlit run src/app.py
# Приложение открывается на http://localhost:8501
```

RCPSP Scheduler

Каркас планировщика с подключаемыми эвристиками. Выберите параметры задачи, загрузите данные и запустите решение.

Параметры задачи

Тип задачи: RCPSP (base)

Целевая метрика: Makespan

Тип ресурсов: Renewable

Режим выполнения: Single-Mode

> Дополнительные флаги

☐ Подробный лог выполнения

Данные инстанса

Загрузите JSON-файл с данными проекта (поля resources и jobs).

Выберите файл инстанса (.json)

Upload 200MB per file • JSON

DSL-описание (опционально)

Если не загружен, DSL будет сформирован автоматически из выбранных параметров выше.

Выберите файл DSL (.json)

Upload 200MB per file • JSON

Встроенная эвристика Вставить код от LLM

Функция выбора задачи (select_job_fn): Most Successors (default)

Запустить планировщик

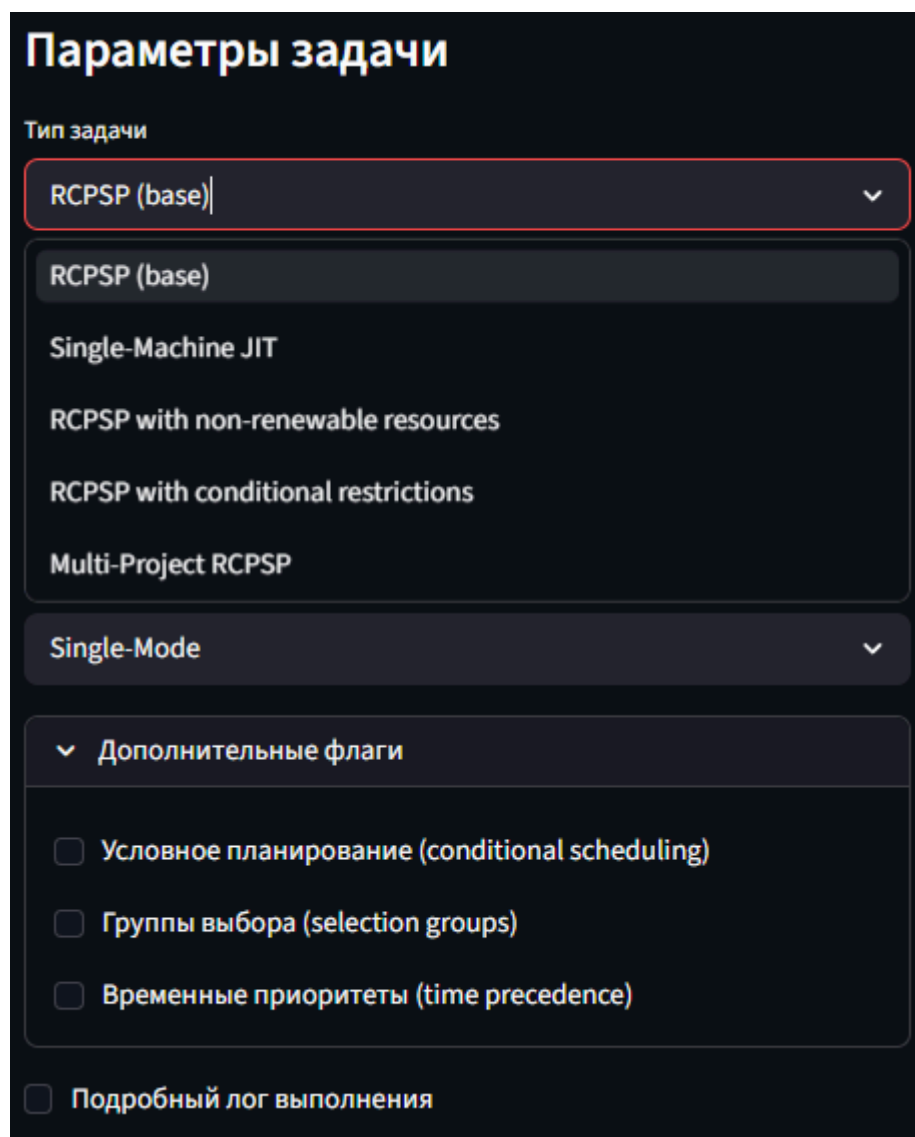
Рис. 1. Главное окно веб-интерфейса системы планирования на базе Streamlit.

Интерфейс использует `scheduler_skeleton.py` напрямую через публичный API: `build_state(dsl_path, instance_data)` и `run_schedule(state,`

select_fn, allocate_fn). Это обеспечивает полную эквивалентность результатов между веб-интерфейсом и скриптами.

5.2. Структура интерфейса

Интерфейс разделен на три зоны. Левая колонка (Параметры задачи), представленная на рисунке 2, содержит выбор типа постановки из пяти вариантов, целевой метрики, типа ресурсов, режима выполнения и дополнительных флагов условного планирования.



Параметры задачи

Тип задачи

RCPSP (base) ▼

RCPSP (base)

Single-Machine JIT

RCPSP with non-renewable resources

RCPSP with conditional restrictions

Multi-Project RCPSP

Single-Mode ▼

▼ Дополнительные флаги

☐ Условное планирование (conditional scheduling)

☐ Группы выбора (selection groups)

☐ Временные приоритеты (time precedence)

☐ Подробный лог выполнения

Рис. 2. Панель выбора конфигурации задачи и дополнительных ограничений.

Правая колонка (Данные инстанса) – загрузку JSON-файла задачи и опциональную загрузку DSL-файла (рис. 3). Если DSL не загружен, он формируется автоматически из параметров левой колонки.

Данные инстанса

Загрузите JSON-файл с данными проекта (поля `resources` и `jobs`).

Выберите файл инстанса (.json)

Case1_...j301_1.json 8.7KB +

DSL-описание (опционально)

Если не загружен, DSL будет сформирован автоматически из выбранных параметров выше.

Выберите файл DSL (.json)

Case1_DSL.json 1.7KB +

Файл загружен: 32 работ, 4 ресурсов

▼ Предпросмотр данных (первые 3 работы)

```
[
  {
    "id": 1
    "predecessors": []
    "successors": [
      2
      3
      4
    ]
  }
]
```

Рис. 3. Интерфейс загрузки входных данных инстанса и DSL-описания задачи.

Нижняя зона содержит две вкладки:

```
tab_builtin, tab_llm = st.tabs([
    "Встроенная эвристика",
    "Вставить код от LLM",
])
```

Вкладка «Встроенная эвристика» (рис. 4) предлагает четыре встроенных правила приоритета: Most Successors (работа с наибольшим числом преемников), SPT (кратчайшее время выполнения), LPT (наибольшее время выполнения), MSLK (минимальный запас хода – разница между ранним стартом и текущим временем). Встроенные функции компилируются внутри app.py по тому же интерфейсу, что и функции от LLM.

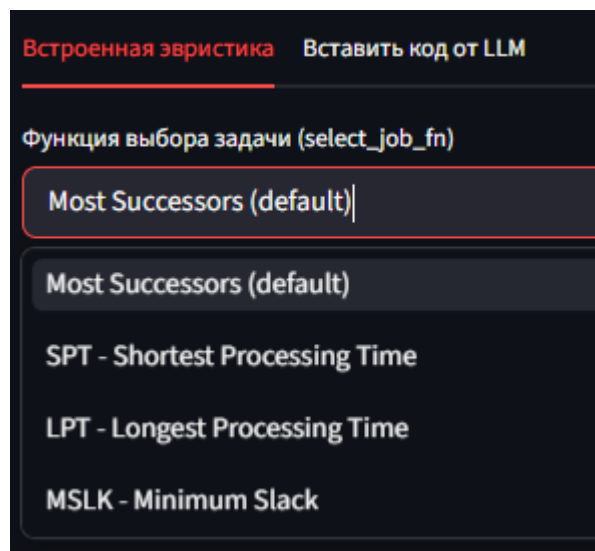


Рис. 4. Окно выбора встроенных правил приоритетов.

Вкладка «Вставить код от LLM» содержит текстовое поле для `select_job_fn` и `allocate_fn`, кнопки «Проверить синтаксис» и «Запустить с этим кодом», а также кнопку справки с описанием доступных полей `Job`, `Resource` и `snapshot` (рис. 5-6).

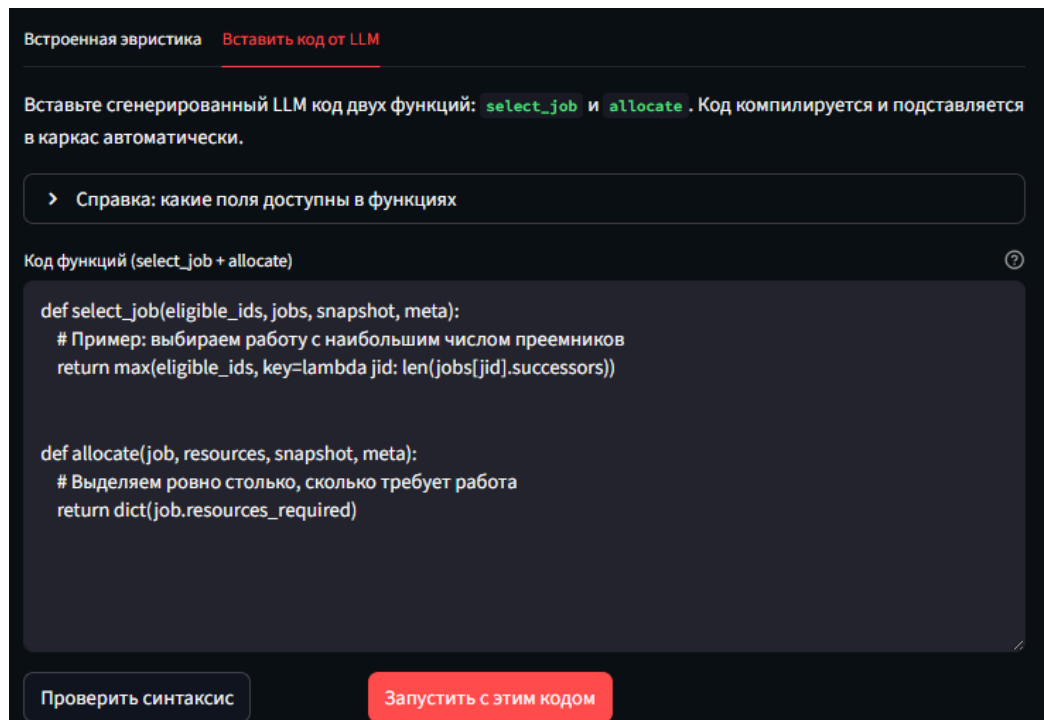


Рис. 5. Форма ввода и интерактивной отладки пользовательского кода эвристик.

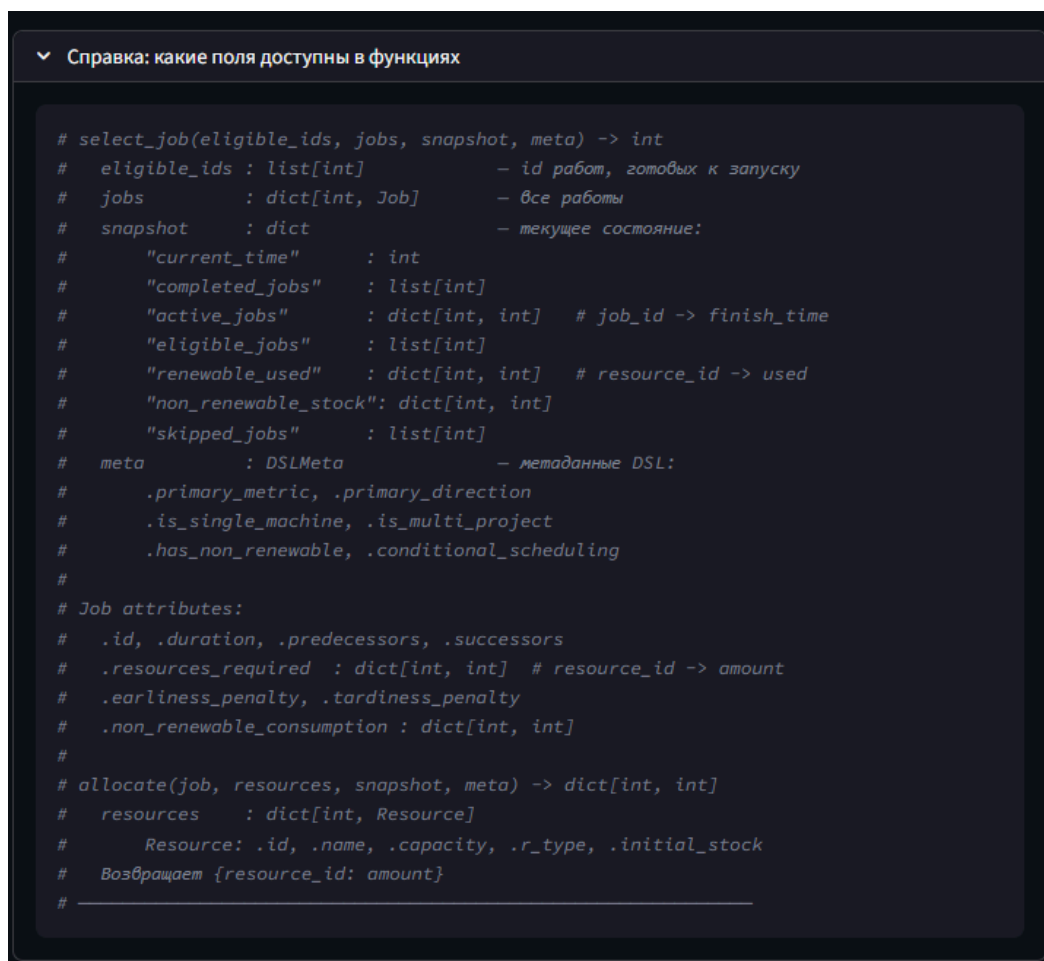


Рис. 6. Справочная информация доступных атрибутов.

5.3. Компиляция кода от LLM

Код от LLM компилируется через `exec()` в изолированный модуль. Функция `compile_fn` в `app.py` полностью аналогична той, что используется в скриптах `run_heuristics_*.py`:

```
def compile_fn(code: str, fn_name: str):
    mod = types.ModuleType('llm_heuristic')
    exec(compile(code, '<llm_heuristic>', 'exec'), mod.__dict__)
    fn = getattr(mod, fn_name, None)
    if fn is None:
        raise ValueError(f'Функция {fn_name!r} не найдена в коде')
    return fn
```

Кнопка «Проверить синтаксис» вызывает `compile_fn` без запуска планировщика и выводит либо «ОК», либо текст исключения с трассировкой. Это позволяет итеративно отлаживать код прямо в интерфейсе.

5.4. Отображение результатов

По завершении планирования интерфейс отображает четыре метрики в колонках: значение целевой функции, число запланированных работ, таблицу расписания с колонками Job ID / Start / Duration / End, диаграмму Ганта на базе Altair и полный JSON результата (Рис. 7-8). Еще доступна кнопка для скачивания расписания.

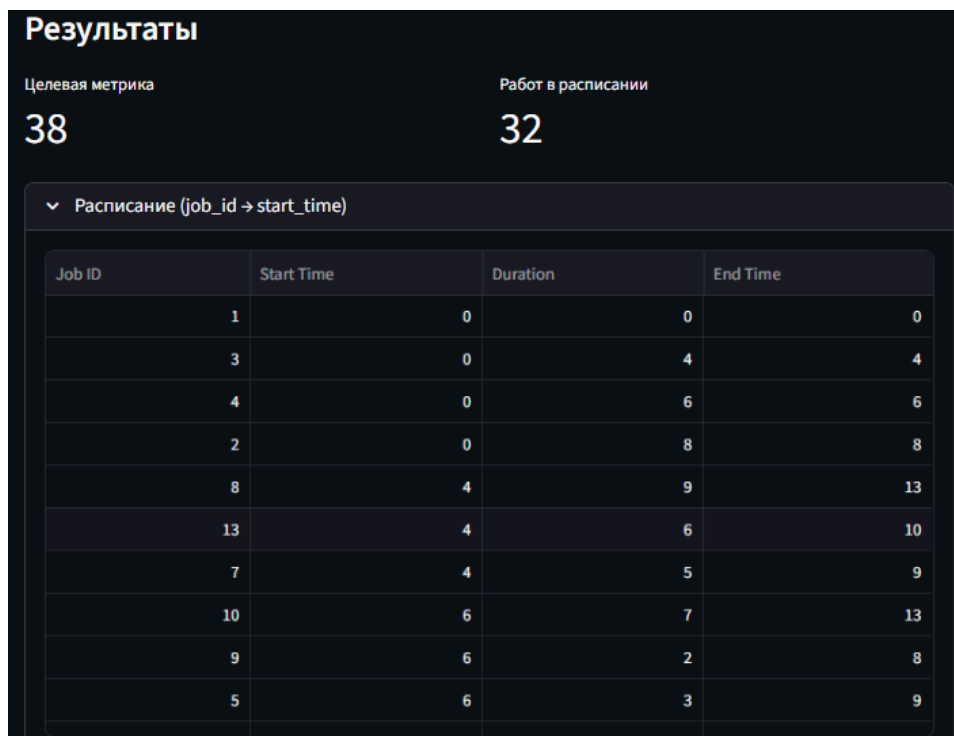
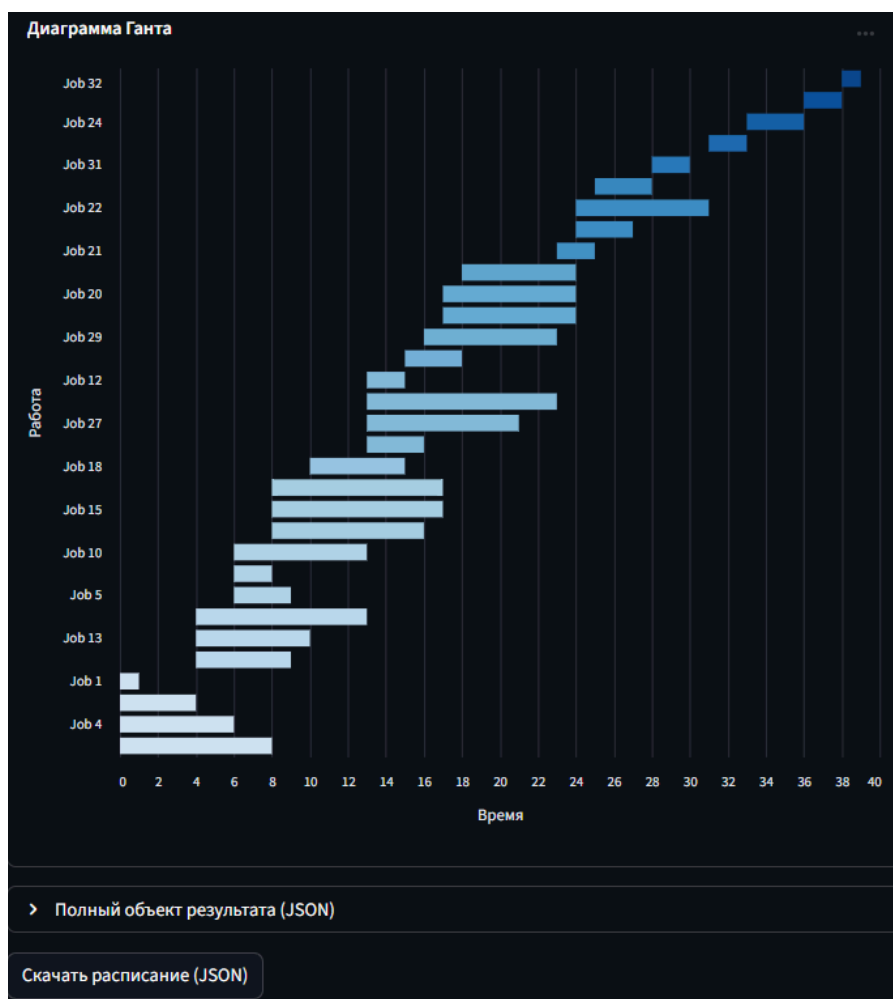


Рис. 7. Панель вывода результирующих метрик и таблицы расписания



5.5. Выводы по главе

В данной главе представлен разработанный веб-интерфейс системы, реализованный на базе фреймворка Streamlit и предназначенный для интерактивного решения задач календарного планирования. Архитектура приложения построена таким образом, чтобы обеспечить полную эквивалентность результатов между веб-интерфейсом и консольными скриптами за счет использования единого публичного API модуля `scheduler_skeleton.py`.

Глава 6. Экспериментальное исследование

6.1. Условия экспериментов

Все три подхода тестировались на одинаковом наборе из пяти кейсов с использованием модели gpt-5.4-nano через OpenAI-совместимый API. Данная версия nano была выбрана для симуляции жестких ограничений по вычислительному бюджету и проверки способности малых моделей к строгому следованию формату. Параметры генерации единые для всех подходов: temperature=0.7, max_tokens от 5000 до 8000, 10 запусков на кейс. Итого было выполнено 150 запросов к LLM.

Скрипты генерации: генерации включали отдельные клиенты для текстовых описаний, DSL-описаний и Skeleton-каркаса. Результаты тестирования сохранялись и агрегировались для последующего анализа метрик и токенов.

6.2. Токены: сравнение диалог-стоимости

Анализ расхода токенов показал (рис. 9), что подход Skeleton демонстрирует существенное преимущество по объему генерируемого ответа. Completion Tokens – является ключевым показателем вычислительной нагрузки и прямым фактором стоимости при использовании LLM. Чем меньше токенов генерирует модель при сохранении корректности решения, тем эффективнее подход с точки зрения ресурсопотребления.

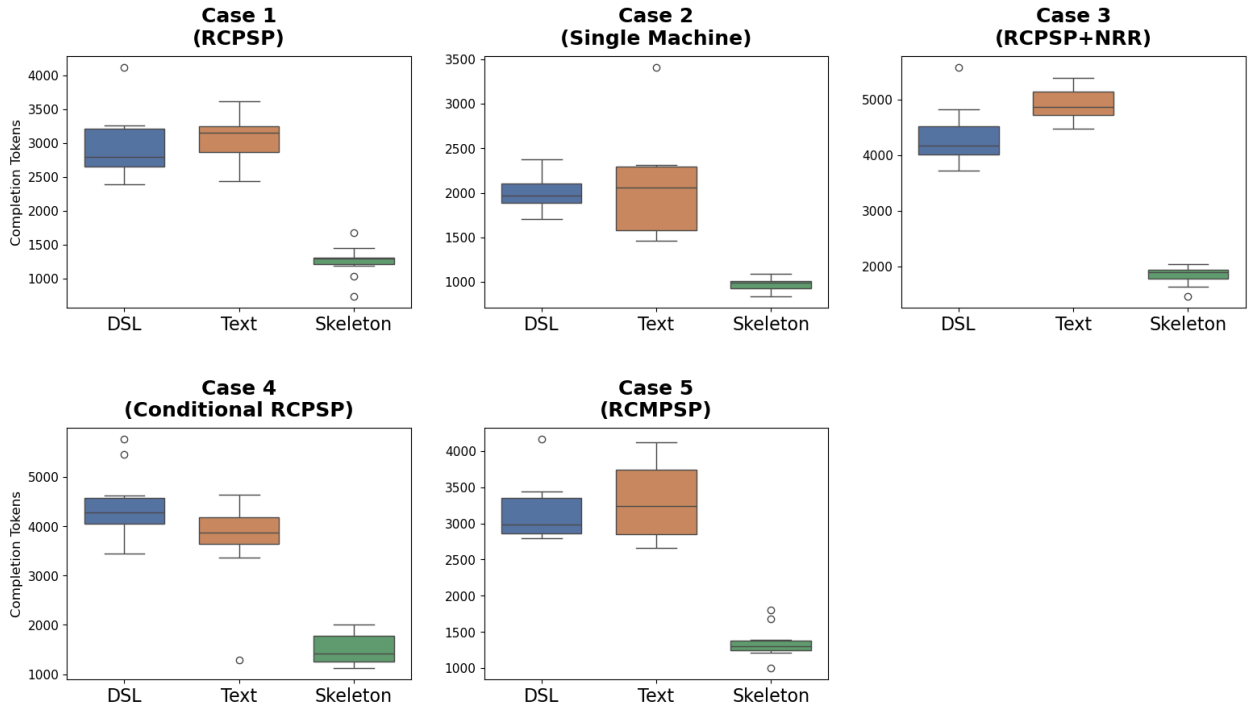


Рис. 9. Диаграммы размаха количества выходных токенов
в разрезе подходов и типов задач.

Среднее количество выходных токенов варьируется от 973 до 1838 в зависимости от задачи, что в 1,5-3 раза меньше по сравнению с текстовым подходом (2072-4920 токенов) и DSL (2007-4427 токенов). Сокращение объема генерации достигает 51-66%.

6.3. Надежность: процент успешных запусков

Надежность оценивалась как доля запусков, завершившихся генерацией корректно работающего кода без системных ошибок. На рисунке 10 видно, что Skeleton показал наивысшую стабильность, а именно 48 успешных запусков из 50. Единственным типом ошибок стали редкие сбои в генерации валидного формата JSON.

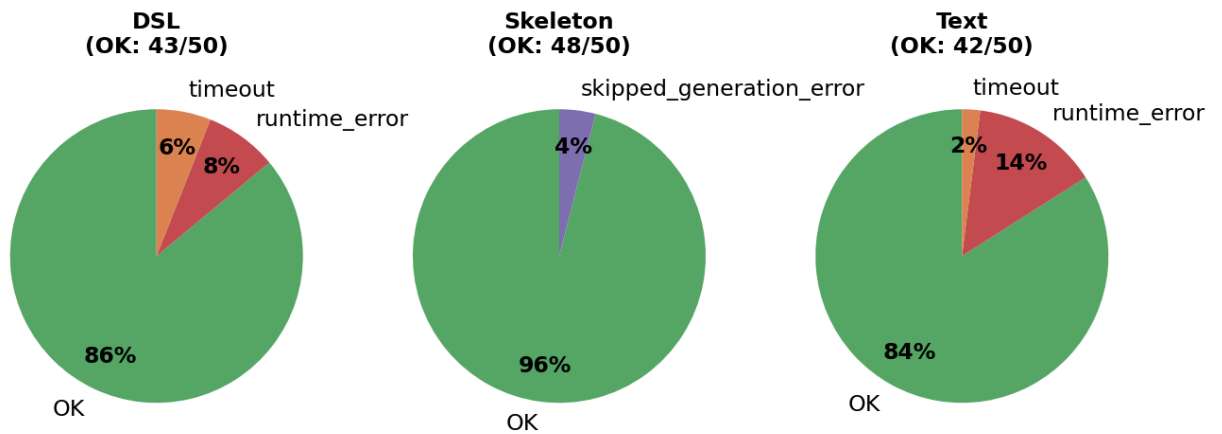


Рис. 10. Распределение статусов компиляции и выполнения сгенерированных решений по подходам.

Подходы DSL и Text показали надежность на уровне 84-86%. Но профиль их ошибок оказался более критичным:

- Для DSL основными проблемами стали ошибки выполнения кода (`runtime_error`) и превышение лимита времени (`timeout`), особенно в кейсах с условным планированием и многопроектной структурой, где монолитный код уходил в бесконечный цикл.
- Текстовый метод оказался наименее стабильным в задачах со сложными ограничениями, показав лишь 60% успешных запусков для четвертого кейса.

6.4. Качество решений

Анализ качества решений проводился только на успешных запусках, которые вернули валидное значение целевой метрики. Это позволило исключить влияние ошибок выполнения на статистику.

Для каждого кейса сравнивалась медиана, среднее значение и стандартное отклонение целевого результата. Там, где известна нижняя граница или оптимальное решение задачи, результаты дополнительно соотносились с этим эталоном.

6.4.1. Базовый RCPSP

Целевая метрика – минимизация makespan (общее время завершения).

Skeleton продемонстрировал абсолютный результат: во всех 10 запусках было получено значение метрики 38, что соответствует известному оптимуму задачи (рис. 11-12).

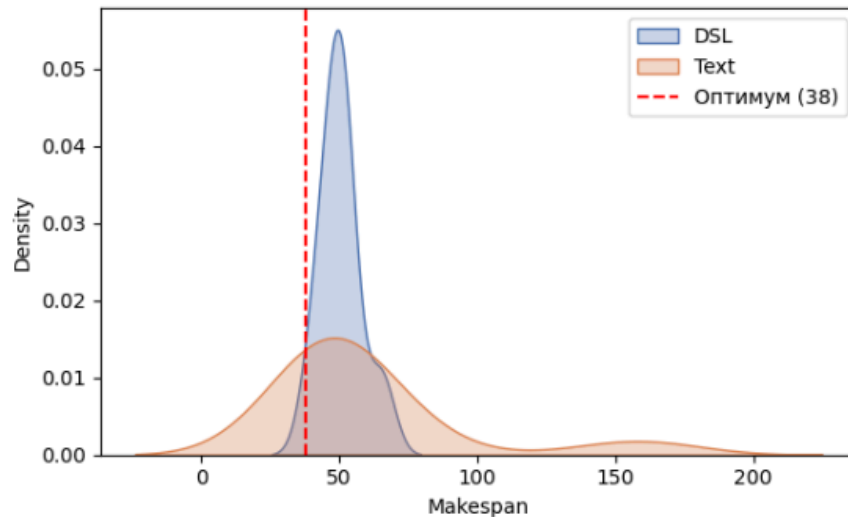


Рис. 11. Плотность распределения значений целевой метрики для базовой задачи RCPSP.

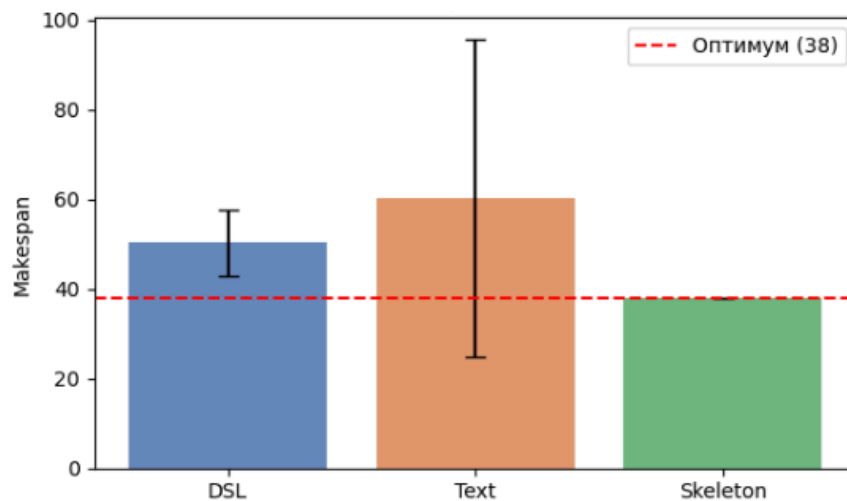


Рис. 12. Сравнение средних значений и стандартных отклонений makespan для базовой задачи RCPSP.

DSL показал среднее значение 50,2 при значительном разбросе. Результаты текстового представления задачи оказались наименее точными.

Среднее значение составило 60,3, а распределение значений отличается выраженными выбросами, что указывает на нестабильность генерируемых эвристик.

6.4.2. Single Machine / JIT

Целевая метрика – минимизация суммарного штрафа за отклонение от директивных сроков. Эталонная оценка была получена с помощью V-shape эвристики, которая составляет 92760.

Данный кейс стал системно проблемным для подходов DSL и Text. На рисунке ниже показано, что генерируемый ими код вычислял метрику некорректно, возвращая значения ниже физически возможного минимума. Это свидетельствует именно о фундаментальной ошибке в выборе и реализации функции планирования.

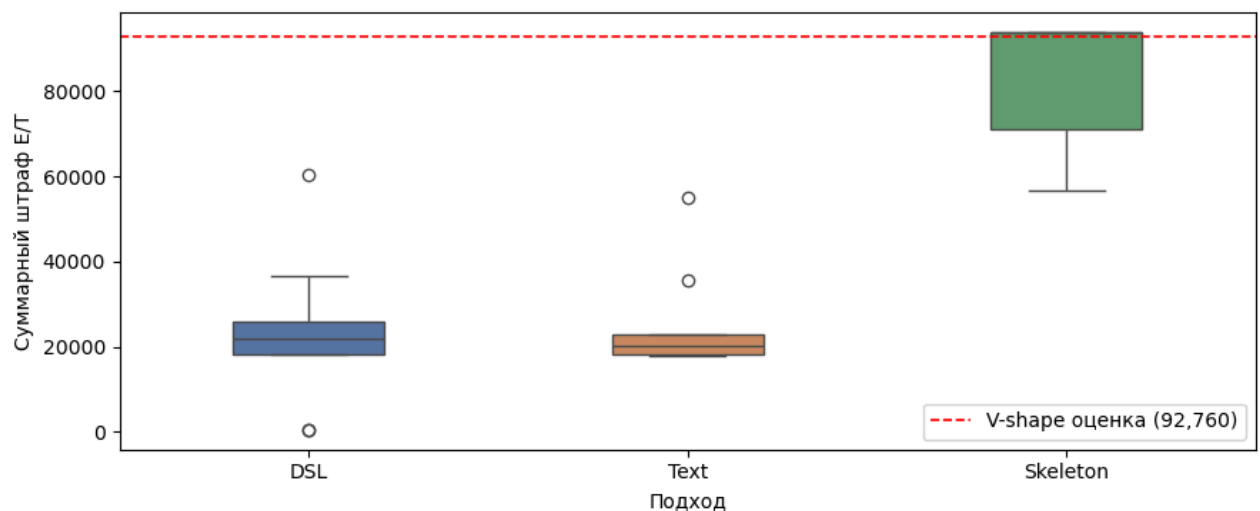


Рис. 13. Диаграмма размаха значений суммарного штрафа для задачи Single Machine.

Skeleton получился единственным подходом, обеспечивающим более достоверные результаты. Благодаря жесткой структуре каркаса, который сам реализует логику вычисления штрафа, модель была избавлена от необходимости корректно интерпретировать сложную структуру целевой функции.

6.4.3. RCPSP с невозобновляемыми ресурсами и условный RCPSP

Целевая метрика у этих инстансов – минимизация makespan. Третий кейс включает выбор альтернативных ветвей по невозобновляемым ресурсам, а кейс четыре – условное планирование с OR-узлами.

Важно отметить методологическую особенность оценки данных задач: нижняя граница CPL (длина критического пути без учета ресурсных ограничений) рассчитывалась по полному исходному графу, содержащему абсолютно все потенциальные альтернативы и избыточные технологические связи. Поскольку в процессе корректного планирования выбираются только конкретные ветви, а неиспользуемые подграфы полностью отсекаются, реальный исполняемый граф работ оказывается значительно короче исходного монолитного графа. По этой причине результирующие значения makespan для корректных расписаний оказываются существенно ниже теоретического значения CPL. В данном контексте CPL полного графа служит лишь верхним структурным референсом, а эффективность алгоритмов оценивается путем сопоставления полученных результатов между собой.

Каркасный подход показал идеальную стабильность, а именно makespan составили 24,2 и 15,2 для третьего и четвертого задач соответственно, при практически нулевом стандартном отклонении (рис. 14-15). Это означает, что во всех успешных запусках модель строила практически идентичные расписания.

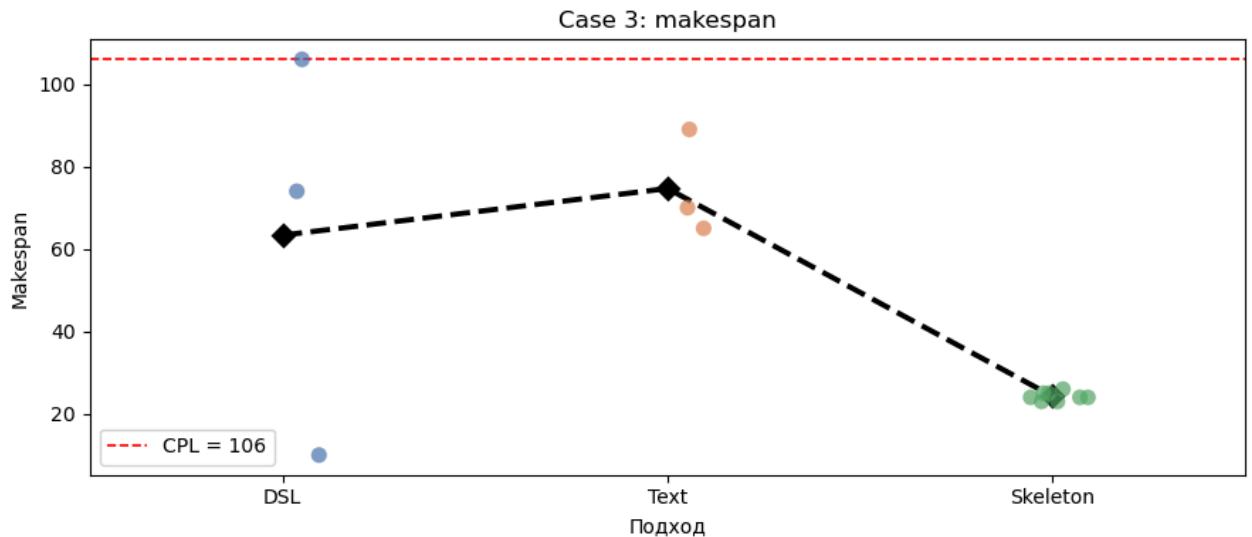


Рис. 14. Сравнение полученных значений makespan для задачи RCPSP с невозобновляемыми ресурсами.

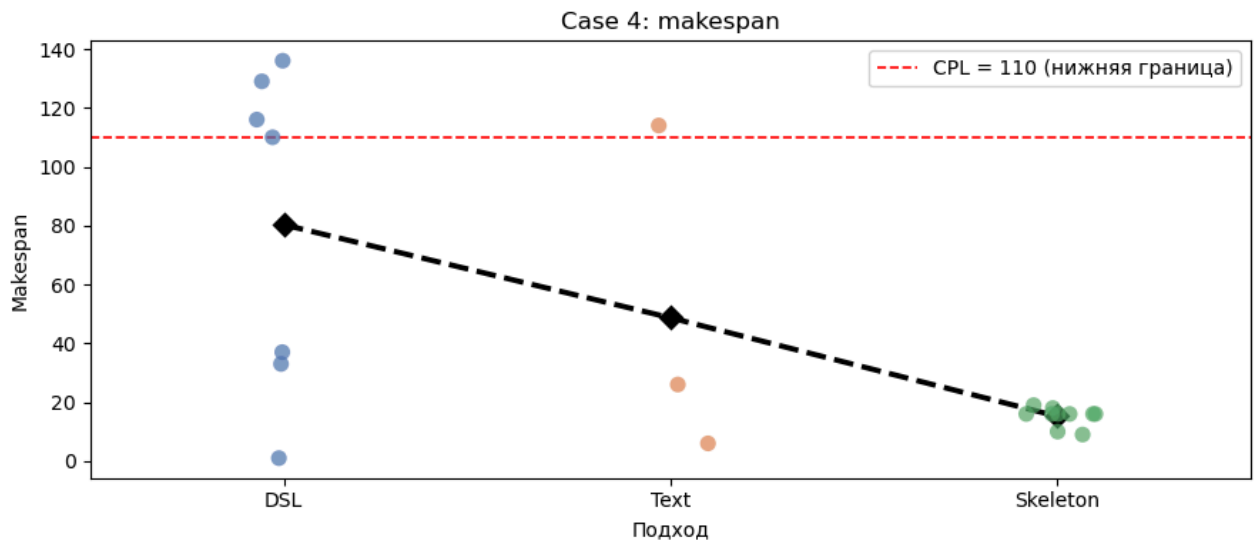


Рис. 15. Сравнение полученных значений makespan для задачи с условным планированием.

Монолитные реализации обладали принципиальным недостатком архитектуры, то есть генерируемый ими код планировал избыточные ветви, не учитывая условную логику задачи. Это приводило к систематическому завышению метрики и, что важно, к значительному разбросу результатов. Стандартное отклонения достигли 57,5 единиц, что делает результаты этих подходов ненадежными для практического применения.

6.4.4. Многопроектный RCPSP

Целевая метрика – минимизация суммарного makespan всех проектов. Известный оптимум составляет 88.

Все три подхода приближены к оптимальному значению (рис. 16), но имеют существенно разные результаты надежности.

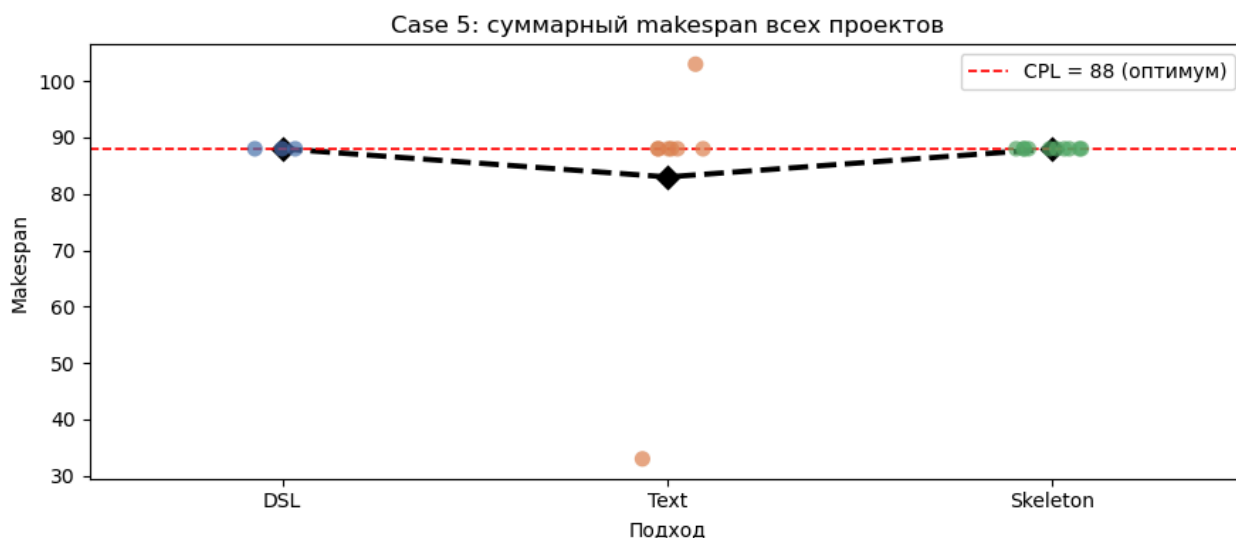


Рис. 16. Сравнение значений суммарного makespan для многопроектной задачи RCPSP.

Skeleton выполнил корректно все 10 из 10 запусков и во всех достиг значения makespan = 88. DSL завершил без ошибок только 3 запуска из 10; в успешных запусках качество решений оказалось сопоставимым со Skeleton, но низкая надежность нивелирует это преимущество. Text занял промежуточное положение по надежности, но уступил каркасному решению по стабильности значений целевой функции.

6.5. Анализ генерируемых эвристик

Исследование показало разницу в стиле выбора методов решения задач планирования в зависимости от подхода. Во всех случаях LLM использует жадные алгоритмы, базирующиеся на схемах генерации расписаний, но реализуют их по-разному, а именно:

- Текстовый подход не накладывал жестких структурных ограничений на генерацию, что приводило к попыткам модели применить сложные метаэвристики (Multi-start и GRASP), которые трудно реализовать правильно в ограниченном контексте. Это, вероятно, привело к обрезке кода и ошибкам.
- DSL-описания проектов заставил модель генерировать решения с классическими названиями из литературы по исследованию операций (Serial SGS с приоритетом по критическому пути). Но при создании монолитного кода LLM допускала ошибки в инфраструктурных частях, выдавая, например, бесконечные циклы.
- Каркасная реализация сфокусировала модель на инженерной логике. Избегая сложных метаэвристик, LLM написала детерминированные правила разрешения коллизий и точного распределения ресурсов. Сложность задачи решалась не через рандомизацию, а через встраивание жестких проверок (давление дефицита невозобновляемых ресурсов) прямо в функцию приоритета.

6.6. Выводы по главе

Архитектурное преимущество разработанного каркасного подхода подтвердилось в ходе экспериментов. Изоляция логики всего цикла решателя задач планирования от эвристической логики решила сразу несколько системных проблем монолитной генерации:

1. Исключение бесконечных циклов;
2. Исключение ошибок расчета целевой функции;
3. Снижение диалог-стоимости;
4. Обеспечение надежности генерации валидного кода.

Глава 7. План дальнейшей разработки

Для успешного применения созданного инструмента в реальных индустриальных задачах менеджмента требуется расширение математической модели и функциональных возможностей библиотеки.

Во-первых, в текущей версии библиотеки реализовано классическое отношение предшествования типа “финиш-старт” с нулевой временной задержкой. Для моделирования сложных производственных циклов необходимо внедрение обобщенных связей предшествования (Generalized Precedence Relations).

Во-вторых, нужен переход от классической задачи RCPSP к многорежимной постановке (Multi-Mode RCPSP). В рамках Skeleton-интерфейса это потребует расширения зоны ответственности LLM. Модель должна будет генерировать код, который вычисляет не только приоритет работы, но и одновременно выбирает оптимальный режим выполнения для этой работы на основе текущего состояния фронта работ и дефицита ресурсов.

В-третьих, на практике для выполнения работы часто требуется не один тип ресурсов, а именно их набор. Введение мульти-ресурсных требований усложнит процедуру последовательного распределения ресурсов (SSGS) в детерминированном ядре `scheduler_skeleton.py`.

Заключение

В ходе выполнения технологического проекта была разработана и экспериментально исследована программная библиотека генерации эвристик для задач планирования на основе больших языковых моделей. В основу работы легла предложенная концепция Skeleton-интерфейса, которая базируется на принципе строгого разделения ответственности между детерминированным вычислительным ядром и языковой моделью.

Основные результаты работы:

1. Разработан универсальный каркас планировщика на языке Python (модуль `scheduler_skeleton.py`), реализующий последовательную схему генерации расписаний (SSGS), учет возобновляемых и невозобновляемых ресурсов;
2. Реализован интерактивный веб-интерфейс на базе фреймворка Streamlit, предоставляющий пользователю инструментарий для формирования параметров и данных проекта планирования и визуализацию результатов в виде таблицы расписания и диаграммы Ганта;
3. Проведено комплексное экспериментальное исследование трех подходов к взаимодействию с LLM (текстовый промпт, полное DSL-описание и каркасный Skeleton-интерфейс). На основе 150 выполненных запросов к модели gpt-5.4-nano доказано статистическое преимущество предложенного каркасного подхода:
 - Объем выходных токенов сократился на 51-66% по сравнению с монолитными аналогами;
 - Показатель надежности вырос до 96%;
 - Устранены дефекты, связанные с бесконечными циклами в коде и семантическими ошибками при расчете метрик.

4. Подтверждено высокое качество генерируемых решений. В базовом сценарии RCPSP подход Skeleton во всех запусках обеспечил точное достижение известного глобального оптимума, а в задачах с условной топологией продемонстрировал минимальную дисперсию результатов.

Таким образом, разработанный подход интеграции LLM в алгоритмический каркас имеет высокую практическую значимость для автоматизации проектирования систем управления производственными процессами и может масштабироваться на более широкие классы задач комбинаторной оптимизации.

Список использованных источников

1. Wang R., Huang Y., Li M. et al. Rethinking LLM-Driven Heuristic Design: Generating Efficient and Specialized Solvers via Dynamics-Aware Optimization [Электронный ресурс]. — 2025. — URL: <https://arxiv.org/abs/2601.20868> (дата обращения: 07.06.2026).
2. Ye H., Wang J., Cao Z. et al. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution [Электронный ресурс]. — 2024. — URL: <https://arxiv.org/abs/2402.01145> (дата обращения: 07.06.2026).
3. Corrêa A. B., Pereira A. G., Seipp J. Classical Planning with LLM-Generated Heuristics: Challenging the State of the Art with Python Code [Электронный ресурс]. — 2025. — URL: <https://arxiv.org/abs/2503.18809> (дата обращения: 07.06.2026).
4. LLM Cost Optimization: 5 Levers to Cut API Spend 70-85% [Электронный ресурс] // Morph. — URL: <https://www.morphllm.com/llm-cost-optimization> (дата обращения: 07.06.2026).
5. OpenAI Platform. Chat interface [Электронный ресурс] // OpenAI. — URL: <https://platform.openai.com> (дата обращения: 07.06.2026).
6. DSL_to_heuristics_process [Электронный ресурс] / Михалева С.Д. — URL: https://github.com/AISheduling/DSL_to_heuristics_process (дата обращения: 07.06.2026).